

Connexion TCP

[Principe TCP](#)

[Sockets TCP en Java](#)

[Exemples de code complets](#)

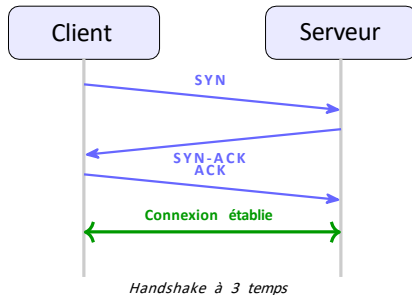
[Critique de TCP](#)

[Comparaison TCP / UDP et choix de protocole](#)

TCP : mode connecté et flux de données

Mode connecté = une connexion avant tout échange

- ▶ Avant d'envoyer des données, les deux parties **établissent une connexion** (handshake à 3 temps).
- ▶ La connexion crée un **canal virtuel bidirectionnel** entre client et serveur.
- ▶ Les données circulent comme un **flux continu d'octets** (*byte stream*), sans frontière de message.
- ▶ Analogie : **appel téléphonique** – on décroche, on parle, on raccroche.



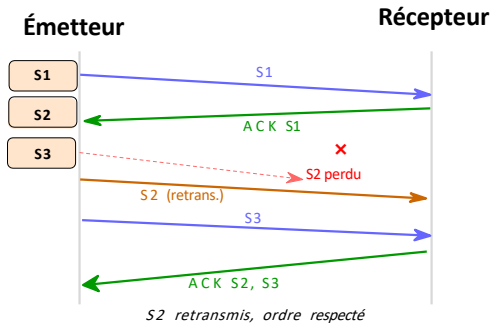
TCP : fiabilité et ordre de livraison

Fiabilité garantie

- ▶ Chaque segment reçoit un **accusé de réception (ACK)**.
- ▶ Tout segment non acquitté est **retransmis automatiquement**.
- ▶ Aucune donnée n'est perdue sans que l'application le sache.

Ordre de livraison respecté

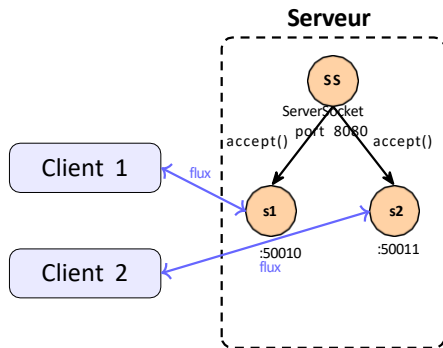
- ▶ TCP réordonne les segments à l'arrivée.
- ▶ L'application reçoit les données **dans le même ordre** qu'elles ont été envoyées.



TCP côté serveur : socket d'écoute et socket de service

Socket d'écoute (ServerSocket)

- ▶ Liée au **port d'écoute fixe** du serveur.
- ▶ **Attend** les demandes de connexion entrantes via `accept()`.
- ▶ Ne sert **jamais** à échanger des données.
- ▶ Reste ouverte tout au long de la vie du serveur.

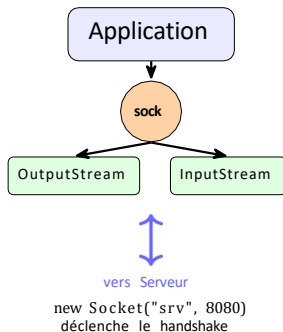


Socket de service (Socket)

- ▶ Créée par `accept()` pour **chaque client**.
- ▶ Porte un **port éphémère** différent du port d'écoute.
- ▶ C'est via cette socket que s'effectue l'**échange réel**.
- ▶ Fermée quand la communication avec ce client est terminée.

TCP côté client : rôle de la Socket

- ▶ Le client crée une Socket en précisant l'**adresse IP** et le **port** du serveur.
- ▶ La création de la socket **déclenche automatiquement** le handshake TCP.
- ▶ Si le serveur n'est pas disponible ⇒ `ConnectException` levée.
- ▶ Une fois connecté, le client accède à deux flux :
 - ▶ `OutputStream` – écriture vers le serveur
 - ▶ `InputStream` – lecture depuis le serveur
- ▶ Le client **initie** toujours la connexion.



InetAddress – rappel

Méthodes de création (statiques)

- ▶ `InetAddress.getByName("192.168.1.10")`
- ▶ `InetAddress.getByName("monserveur.local")`
- ▶ `InetAddress.getLocalHost()`
Toutes lèvent `UnknownHostException`.

Méthodes d'interrogation

- ▶ `getHostAddress()` →
"192.168.1.10"
- ▶ `getHostName()` → "monserveur"

En TCP : usage simplifié

En pratique, avec `Socket`, on passe souvent directement le nom de machine en `String` sans créer explicitement un `InetAddress` :

- ▶ `new Socket("localhost", 8080)`
- ▶ `new Socket("192.168.1.20", 8080)`

Java résout le nom en interne.

- ▶ `InetAddress` reste utile pour **interroger l'adresse du client** côté serveur :
`socket.getInetAddress()`

La classe Socket– constructeurs et méthodes

Constructeurs :

```
// Par nom de machine (résolution DNS
// interne)
Socket s = new Socket("localhost",
    8080);

// Par adresse IP explicite
Socket s = new Socket("192.168.1.20",
    8080);

// Par InetAddress
InetAddress adr = InetAddress.getByName
    ("srv");
Socket s = new Socket(adr, 8080);
```

Récupération des flux :

```
InputStream in = s.getInputStream();
OutputStream out = s.getOutputStream();
```

Méthodes d'information :

getPort()	port distant (serveur)
getLocalPort()	port local (client)
getInetAddress()	IP distante

Timeout et fermeture :

```
// Timeout de lecture en ms
s.setSoTimeout(3000);

// Fermeture (libère ressources)
s.close();
```

Exception à gérer

Constructeur lève IOException
(ou ConnectException si serveur absent).

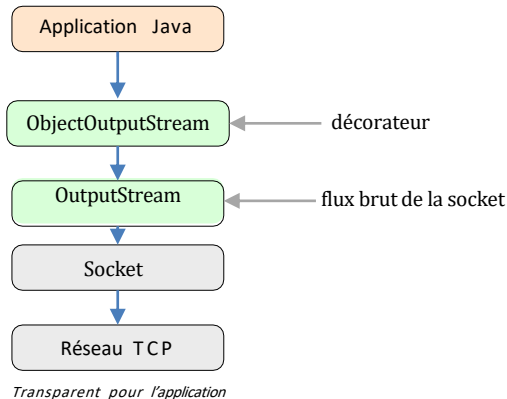
Flux objets et texte sur TCP : décorer les flux de la socket

Flux d'objets Java (Serializable) :

```
// Émission
ObjectOutputStream oos =
    new ObjectOutputStream(s.getOutputStream());

oos.writeObject(monObjet);
oos.flush();

// Réception
ObjectInputStream ois =
    new ObjectInputStream(s.getInputStream());
Object obj = ois.readObject();
```



Flux texte (lecture ligne par ligne) :

```
// Lecture
BufferedReader br = new BufferedReader(new InputStreamReader(s.getInputStream()));
```


La classe ServerSocket– écoute et acceptation

Constructeur :

```
// Lié au port d'écoute  
ServerSocket ss = new ServerSocket  
    (8080);
```

Accepter une connexion :

```
// Bloque jusqu'à l'arrivée d'un client  
// Retourne une Socket de service  
Socket client = ss.accept();
```

Timeout sur accept() :

```
// Délai max d'attente (ms)  
ss.setSoTimeout(5000);  
// Lève SocketTimeoutException si expiré
```

Fermeture :

```
ss.close(); // libère le port d'écoute
```

Rôle de accept ()

- ▶ **Bloque** tant qu'aucun client ne se connecte.
- ▶ Quand un client arrive : effectue le handshake TCP.
- ▶ Retourne une Socket de service dédiée à ce seul client.
- ▶ Le ServerSocket continue d'écouter sur le même port.

Boucle serveur typique

```
ServerSocket ss = new ServerSocket(8080);  
while (true) {  
    Socket client = ss.accept(); // traiter client ...  
    client.close();}
```

Client TCP (1/2) – envoi d'un objet

```
1  import java.net.*;
2  import java.io.*;
3
4  public class ClientTCP {
5      public static void main(String[] args) throws Exception {
6          // 1. Connexion au serveur (déclenche le handshake TCP)
7          Socket socket = new Socket("localhost", 8080);
8          System.out.println("Connecté au serveur "
9              + socket.getInetAddress() + ":" + socket.getPort());
10
11         // 2. Décorer les flux pour travailler avec des objets Java
12         ObjectOutputStream oos =
13             new ObjectOutputStream(socket.getOutputStream());
14         ObjectInputStream ois =
15             new ObjectInputStream(socket.getInputStream());
16
17         // 3. Envoyer un objet (Message doit implémenter Serializable)
18         Message requete = new Message("Bonjour Serveur", 1);
19         oos.writeObject(requete);
20         oos.flush();
21         System.out.println("Requête envoyée : " + requete);
```

Client TCP (2/2) – réception d'une réponse

```
1      // 4. Attendre et lire la réponse du serveur                                socket.  
2      setSoTimeout(5000);                                     // timeout de lecture : 5 s  
3      Object reçu = ois.readObject();  
4  
5      if (reçu instanceof Message) {  
6          Message reponse = (Message) reçu;  
7          System.out.println("Réponse reçue : " + reponse);  
8      } else {  
9          System.out.println("Type inattendu : " + reçu.getClass());  
10     }  
11  
12     // 5. Fermer les flux et la socket  
13     ois.close();  
14     oos.close();  
15     socket.close();  
16 }  
17 }
```

Points importants

- `setSoTimeout()` doit être appelé **avant** `readObject()`.

Serveur TCP correspondant (1/2)

```
1  import java.net.*;
2  import java.io.*;
3
4  public class ServeurTCP {
5      public static void main(String[] args) throws Exception {
6          // 1. Créer la socket d'écoute sur le port 8080
7          ServerSocket serverSocket = new ServerSocket(8080);
8          System.out.println("Serveur TCP en écoute sur le port 8080...");
9
10         // 2. Attendre la connexion d'un client
11         Socket client = serverSocket.accept();
12         System.out.println("Client connecté : "
13             + client.getInetAddress() + ":" + client.getPort());
14
15         // 3. Décorer les flux
16         ObjectInputStream ois =
17             new ObjectInputStream(client.getInputStream());
18         ObjectOutputStream oos =
19             new ObjectOutputStream(client.getOutputStream());
20         // 4. Lire l'objet envoyé par le client
21         Object reçu = ois.readObject();
22     }
```

Serveur TCP correspondant (2/2)

```
1      // 5. Traitement selon le type reçu (instanceof)
2      if (recu instanceof Message) {
3          Message msg = (Message) recu;
4          System.out.println("[Serveur] Message #" + msg.getNumero()
5              + " de " + client.getInetAddress().getHostAddress()
6              + " port " + client.getPort()
7              + " : " + msg.getContenu());
8
9          // 6. Envoyer une réponse
10         Message reponse = new Message("Bien reçu !", msg.getNumero());
11         oos.writeObject(reponse);
12         oos.flush();
13     } else {
14         System.out.println("[Serveur] Type non géré : "
15             + recu.getClass().getName());
16     }
17
18     // 7. Fermer les ressources
19     ois.close();
20     oos.close();
21     client.close();
22     serverSocket.close();
```

instanceof– traiter différents types d'objets reçus

- ▶ Un même flux TCP peut véhiculer **plusieurs types d'objets** (requêtes, commandes, notifications...).
- ▶ Côté récepteur, on lit un Object depuis `ObjectInputStream.readObject()`.
- ▶ On utilise `instanceof` pour **déterminer le type réel** et traiter en conséquence.

Bonne pratique

Toujours inclure un **cas else** pour les types inconnus, afin d'éviter des comportements silencieux.

```
if (recu instanceof Message) {  
    Message m = (Message) recu;  // traiter message}  
else if (recu instanceof Commande) {  
    Commande c = (Commande) recu;  // traiter commande}  
else if (recu instanceof String) {  
    String s = (String) recu;  // traiter chaîne}  
else {  // type non géré  
    System.out.println("Inconnu : " + recu.getClass());}
```

Avantages de TCP

Fiabilité et abstraction

- ▶ **Fiabilité garantie** : aucune perte silencieuse, retransmission automatique.
- ▶ **Ordre respecté** : les données arrivent dans l'ordre d'envoi – aucune gestion côté application.
- ▶ **Contrôle de flux** : le récepteur ne peut pas être saturé.
- ▶ **Contrôle de congestion** : TCP s'adapte à la charge du réseau.

Abstraction via les flux Java

- ▶ Travail avec des flux de haut niveau : `ObjectInputStream`, `PrintWriter`, `BufferedReader`...
- ▶ Envoi **d'objets Java** directement (pas besoin de sérialisation manuelle comme en UDP).
- ▶ L'application ne voit qu'un **tuyau bidirectionnel fiable** – le réseau est transparent.

Cas d'usage classiques :

HTTP/HTTPS, FTP, SSH, bases de données, messagerie (SMTP, IMAP).

Inconvénients de TCP et gestion du multi-client

Complexité accrue

- ▶ **Overhead** : en-tête TCP plus lourd (20 octets min vs 8 pour UDP), mécanismes de contrôle.
- ▶ **Latence** : le handshake ajoute un aller-retour avant tout échange.
- ▶ **Connexion persistante** : ressources occupées pendant toute la durée de la session.

Multi-client = besoin de threads

- ▶ `accept()` bloque le thread principal.
- ▶ Servir plusieurs clients simultanément nécessite de créer **un thread par client**.
- ▶ La socket de service doit être passée au thread pour être traitée de façon indépendante.

Patron typique

```
while (true) {  
    Socket s = ss.accept();  
    new Thread(() -> {  
        traiter(s);  
    }).start();  
}
```


Quand choisir UDP ?

Utiliser UDP quand...

- ▶ La **vitesse** prime sur la fiabilité.
- ▶ La **perte occasionnelle** de données est acceptable (ex. : une image de capteur périmée peut être ignorée).
- ▶ Les messages sont **courts et autonomes**.
- ▶ On a besoin de **broadcast** ou **multicast**.
- ▶ La **latence** doit être minimale.

Quand choisir TCP ?

Utiliser TCP quand...

- ▶ **Toutes les données doivent arriver** sans perte (fichiers, transactions).
- ▶ L'**ordre des données** est essentiel.
- ▶ Les messages sont **longs ou continus** (flux).
- ▶ On utilise des **flux d'objets Java** complexes.
- ▶ La **fiabilité** est plus importante que la performance.

Exemples concrets

- ▶ **NFS (partage de fichiers réseau)** : chaque octet doit être reçu intact.
- ▶ **Transfert de fichiers (FTP)** : intégrité totale requise.
- ▶ **HTTP/HTTPS** : pages web, APIs.
- ▶ **Base de données** distante : requêtes SQL et résultats.
- ▶ **Messagerie** (SMTP, IMAP) : e-mails complets.
- ▶ **SSH** : session interactive, aucune perte tolérée.

Récapitulatif

Principe TCP :

- ▶ Mode connecté, handshake à 3 temps
- ▶ Flux de données fiable et ordonné
- ▶ Socket d'écoute (ServerSocket) + socket de service (Socket)

API Java :

- ▶ Socket : connexion client, flux I/O
- ▶ ServerSocket : écoute, accept()
- ▶ Décoration :
ObjectInputStream/OutputStream
- ▶ instanceof pour typer les objets reçus

Critique TCP :

- + Fiable, ordonné, abstraction flux
- + Envoi d'objets directs
- Overhead, latence (handshake)
- Multi-client $\Rightarrow$ threads

Choix TCP/UDP :

- ▶ UDP : perte tolérable, latence faible, IoT, jeux
- ▶ TCP : fiabilité requise, fichiers, HTTP, NFS

Concurrence dans les systèmes répartis

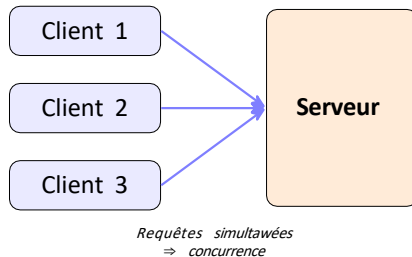
Concurrence dans les systèmes répartis

Spécificités côté serveur T C P

Problème de la gestion simultanée

Les systèmes répartis sont naturellement concurrents

- ▶ Un **système réparti** est composé de plusieurs processus s'exécutant sur des machines différentes et communiquant via le réseau.
- ▶ Les clients n'attendent pas leur tour : les requêtes arrivent **simultanément** et de manière imprévisible.
- ▶ Le serveur doit donc traiter plusieurs clients **en parallèle** – c'est une forme de concurrence.
- ▶ On retrouve ici les mêmes problématiques que dans la programmation concurrentielle locale :
 - ▶ accès à des **ressources partagées**
 - ▶ **synchronisation** entre traitements



Rappel : concurrence locale vs concurrence réseau

Critère	Concurrence locale (threads)	Concurrence réseau (sockets)
Lieu d'exécution	Même JVM, même machine	Machines différentes
Communication	Mémoire partagée	Flux réseau (sockets)
Synchronisation	synchronized, wait/notify	Protocoles, séquençage des messages
Ressource partagée	Objet Java en mémoire	Fichier, base de données, état serveur
Outil Java	Thread, Runnable	Socket + Thread

Ce qui ne change pas

Le serveur TCP multi-clients utilise des threads pour gérer la concurrence réseau, exactement comme pour la concurrence locale. Les deux mondes se combinent.

Les deux types de sockets côté serveur – rappel

ServerSocket – socket d'écoute

- ▶ Liée au **port fixe** connu des clients.
- ▶ Son unique rôle : **attendre** les demandes de connexion via `accept()`.
- ▶ **Ne sert jamais** à échanger des données.
- ▶ Reste ouverte pendant toute la durée de vie du serveur.

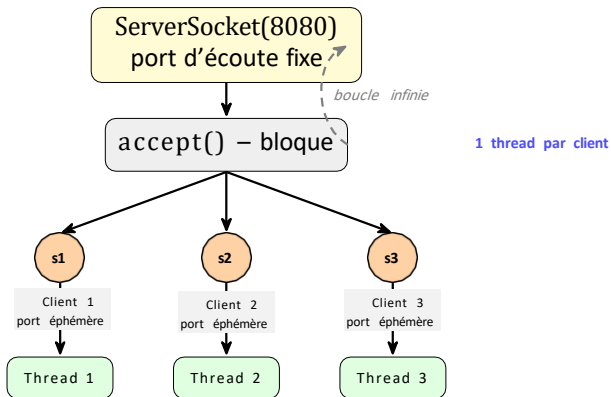
Socket – socket de service

- ▶ **Créée automatiquement** par `accept()` à chaque nouvelle connexion.
- ▶ Dédiée à **un seul client**.
- ▶ Porte un **port éphémère** distinct.
- ▶ C'est elle qui transporte les données.
- ▶ Fermée quand la session avec ce client se termine.

Conséquence directe

Pour servir N clients simultanément, le serveur a besoin de N sockets de service – une par client – toutes gérées **en parallèle**.

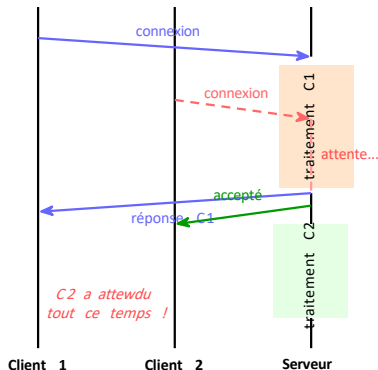
Cycle de vie d'un serveur TCP – vue schématique



Le problème : un serveur séquentiel bloque tout le monde

Serveur séquentiel (naïf)

- ▶ Le serveur accepte un client, **traite sa requête** jusqu'au bout, puis accepte le suivant.
- ▶ Pendant le traitement du client 1, **les autres clients attendent**.
- ▶ Si le traitement est long (calcul, I/O, lecture base de données), le serveur est **inutilisable en pratique**.



Solution : un thread par client

- ▶ Dès qu'accept() retourne une socket, on **délègue son traitement** à un nouveau thread.
- ▶ Le thread principal retourne immédiatement à accept().
- ▶ Les clients sont traités en parallèle.

Patron de base : un thread par client

```
1  ServerSocket ss = new ServerSocket
    (8080);
2
3  while (true) {
4      // Attendre le prochain client
5      Socket client = ss.accept();
6
7      // Déléguer immédiatement à un thread
8      new Thread(() -> {
9          try {
10             traiterClient(client);
11             } catch (Exception e) {
12                 e.printStackTrace();
13             }
14         }).start();
15      // Le thread principal revient à accept()
16  }
```

Ce que fait ce patron

- ▶ `accept()` est **non bloquant pour les autres clients** : dès qu'un client arrive, on lui crée un thread et on continue.
- ▶ Chaque thread gère **sa propre socket de service** de façon indépendante.
- ▶ Le thread principal ne fait que **distribuer** les connexions.

Attention

Si plusieurs threads accèdent à une **ressource partagée** (fichier, compteur, liste etc...), la synchronisation reste nécessaire, comme en programmation concurrentielle locale.

Ressources partagées côté serveur

- ▶ Chaque thread possède **sa propre socket** – pas de partage à ce niveau.
- ▶ En revanche, plusieurs threads peuvent accéder simultanément à des **ressources communes** :
 - ▶ liste des clients connectés
 - ▶ compteur de connexions
 - ▶ fichier journal (*log*)
 - ▶ base de données partagée
- ▶ Ces accès concurrents peuvent causer des **incohérences** sans synchronisation.

